



Cultura
Secretaría de Cultura



ORIGINAL



unesco



PLATAFORMA
ATLAS NACIONAL DE
TÉCNICAS DEL ARTE TEXTIL

MANUAL DE ADMINISTRADOR

Contenido

1. Visión general del sistema	3
1.1 Dónde está alojada la plataforma	3
2. Estructura de archivos.....	3
3. Arquitectura del código	4
3.1 Flujo de carga.....	4
3.2 Modelo de datos: el objeto ATLAS.....	5
3.3 Módulos funcionales de main.js.....	6
3.4 Librerías externas	7
4. Estructura de los archivos CSV.....	7
4.1 data_by_technique_id.csv.....	7
4.2 data_by_record_id.csv.....	9
4.3 indice_imagenes.csv.....	10
4.4 Consistencia entre los CSV	10
5. Cómo actualizar el Atlas	11
5.1 Actualizar datos de técnicas.....	11
5.2 Ajustar el subset de técnicas publicadas	11
5.3 Actualizar la clasificación experta	12
5.4 Agregar o actualizar imágenes	12
5.5 Modificar textos del reporte.....	13
6. Datos: arquitectura actual y migración a API	13
6.1 Arquitectura actual: CSV estáticos	13
6.2 Cómo migrar a una API en el futuro	13
6.3 Consideraciones para una API	14
7. Mantenimiento y diagnóstico	15
7.1 Verificar que la plataforma carga correctamente	15
7.2 El mapa no muestra los estados de México	15
7.3 Actualización de librerías.....	15
8. Formularios de contribución (KoboToolbox)	16
9. Sistema de diseño.....	16

1. Visión general del sistema

El Atlas Nacional de Técnicas del Arte Textil es una aplicación web estática compuesta exclusivamente por archivos HTML, CSS, JavaScript y CSV. No cuenta con servidor propio, no utiliza base de datos y no requiere ningún lenguaje de programación del lado del servidor (como PHP o Python en producción).

Todo el procesamiento de datos ocurre en el navegador del usuario: al cargar la página, el navegador descarga los archivos CSV y construye la interfaz completa en memoria. Esto hace que el sistema resulte extremadamente sencillo de alojar y de mantener.

Principio de diseño fundamental: ningún dato está embebido en el código HTML ni en el JavaScript. Toda la información se lee en tiempo de ejecución desde los archivos CSV. Para actualizar el contenido del Atlas basta con reemplazar los CSV en el servidor y recargar la página.

1.1 Dónde está alojada la plataforma

La plataforma se aloja en los servidores de la Oficina de UNESCO en México. La URL pública de acceso es:

<https://www.unesco.org/mexico/atlas-textil>

Para publicar actualizaciones, el administrador debe subir los archivos modificados al servidor de UNESCO México siguiendo el proceso de publicación que define el equipo de tecnología de esa oficina.

2. Estructura de archivos

El proyecto completo se encuentra en una sola carpeta. Todos los archivos deben permanecer en el mismo directorio para que las referencias relativas entre ellos funcionen correctamente.

```
atlas/
├── index.html      <- Plataforma principal
├── main.js         <- Lógica completa de la aplicación
├── styles.css      <- Estilos globales
├── atlas_textil_v2.html <- Reporte "Acerca del Atlas"
├── data_by_technique_id.csv <- Datos por técnica (fuente canónica)
├── data_by_record_id.csv <- Datos por registro individual
├── indice_imagenes.csv <- Índice de imágenes
└── imagenes/       <- Carpeta de fotografías locales
```

Los archivos CSV deben permanecer siempre en la misma carpeta que index.html. Si se mueven o renombran, la plataforma mostrará un error de carga al inicializarse.

Archivo	Función
index.html	Punto de entrada. Contiene la estructura HTML de las seis vistas, el encabezado y los modales. Referencia main.js y styles.css.
main.js	Lógica completa de la aplicación: carga de los CSV, construcción del modelo de datos, renderizado de cada vista e interacciones.
styles.css	Diseño visual completo: paleta de colores, tipografía (Fraunces, DM Sans, Space Mono), layouts, animaciones y componentes.
atlas_textil_v2.html	Reporte standalone de metodología y estadísticas. Se abre como overlay y carga sus propios CSV de manera independiente.
data_by_technique_id.csv	Fuente canónica de datos estructurados por técnica. Una fila equivale a una técnica.
data_by_record_id.csv	Datos cualitativos por registro de campo. Una fila equivale a una ficha levantada en taller.
indice_imagenes.csv	Mapeo entre archivos de imagen y técnicas. Una fila por archivo de imagen.

3. Arquitectura del código

3.1 Flujo de carga

El ciclo de vida completo de la aplicación al abrirse en un navegador es el siguiente:

- El navegador descarga index.html y muestra el overlay de carga.
- index.html carga main.js y styles.css en paralelo.
- main.js ejecuta loadCSVs(), que descarga los tres CSV en paralelo mediante fetch().
- Los CSV se parsean con un parser propio, sin dependencias externas.
- La función buildAtlasFromCSV() construye el modelo de datos unificado ATLAS en memoria.
- Se aplica el filtro global por la columna "incluida" para conservar solo el subconjunto de técnicas validado por el equipo curatorial.
- Se actualizan las estadísticas del encabezado (técnicas, registros, estados).
- Se inicializa la vista activa (mapa por defecto) y se oculta el overlay de carga.
- Las demás vistas se inicializan bajo demanda al momento en que el usuario navega hacia ellas.

3.2 Modelo de datos: el objeto ATLAS

El objeto global ATLAS es la fuente de verdad de toda la aplicación una vez cargada. Su estructura es la siguiente:

```
ATLAS = {
  tecnicas: [ // Array de objetos, uno por técnica
    {
      tecnica: string,      // Nombre canónico
      cat1, cat2, cat3, cat4, // Clasificación experta (4 niveles)
      grupo: string,       // Categoría CAT-N-1
      estados: string[],   // Estados donde se practica
      lenguas: string[],   // Lenguas indígenas asociadas
      municipios: string[], // Municipios (de record_id)
      n_fichas: number,    // Total de registros
      n_mujeres, n_hombres, // Desglose por género
      edad_promedio: number,
      manufactura: { mano, pedal, telar, ... },
      tenido: { plantas, minerales, animales, otro },
      aprendizaje: { madre, abuela, tia, ... },
      ensenanza: { hijas, nietos, estudiantes, ... },
      materiales: string,
      historia: string,    // Texto más extenso de record_id
      significados: string[], // Fragmentos S1-S5
      imagenes: string[],  // Nombres de archivo
      temporalidad: 'antigua' | 'nueva',

      // Validación curatorial
      score_calidad: number, // Riqueza de los reactivos completados
      score_volumen: number, // Bonus logarítmico por número de registros
      score_total: number,  // Suma de los anteriores
      ranking: number,      // Posición global de la técnica
      incluida: boolean     // Si aparece en el subset público
    }
  ],
  estados: { // Mapa estado -> lista de nombres de técnicas
    'Oaxaca': ['Telar de cintura', 'Brocado', ...],
    ...
  }
}
```

A partir de la versión 2.0, el filtro global de la plataforma utiliza la propiedad "incluida" en lugar del antiguo criterio basado en n_fichas. Solamente las técnicas con incluida = true se muestran en las vistas públicas. El detalle de cómo se calcula este valor se expone en la sección 4.1.

3.3 Módulos funcionales de main.js

main.js está organizado en módulos funcionales secuenciales, delimitados por comentarios de separación. Cada módulo se documenta a continuación:

Módulo / función principal	Responsabilidad
parseCSV / parseCSVRow	Parser CSV propio. Maneja campos entre comillas, comas dentro de valores y saltos de línea.
buildAtlasFromCSV	Construye el objeto ATLAS cruzando los tres CSV. Contiene la lógica de qué campo proviene de cuál fuente. Aplica el filtro por la columna "incluida".
loadCSVs	Carga asíncrona en paralelo de los tres CSV. Maneja errores de red y muestra el banner de error en caso de falla.
initMap / stateStyle / onStateClick	Inicializa el mapa Leaflet, aplica estilos coropléticos, y maneja clics y hover de los estados.
initMapFilter / applyMapFilter	Panel de filtros jerárquico fijo a la izquierda del mapa. Construye los chips en cascada y actualiza los estilos del mapa según el filtro activo.
renderStatesList / onStateSelect	Listado de estados del panel derecho del mapa. Maneja la selección y el despliegue de las técnicas correspondientes.
renderCatalog	Renderiza la cuadrícula de tarjetas del catálogo, con búsqueda y filtros por categoría.
openFicha / closeFicha	Modal de Ficha Detalle. Construye dinámicamente el HTML de cada sección, el carrusel de imágenes y el lightbox.
initNetwork / drawNetwork	Grafo de fuerza sobre canvas. Implementación propia sin D3. Maneja zoom, arrastre, resaltado y búsqueda.
initArbol	Árbol jerárquico con expansión progresiva. Se construye dinámicamente desde las columnas CAT-N-* del CSV.
initTenidoView / renderTenidoMatrix	Matriz de técnicas por tipos de teñido, con resaltado por filtro.
initReportCharts	Gráficas del reporte "Acerca del Atlas" usando Chart.js.

3.4 Librerías externas

La plataforma utiliza tres librerías externas, todas cargadas desde CDN públicos:

Librería	Versión y uso
Leaflet.js	1.9.4 — Mapa interactivo y capas GeoJSON de los estados de México.
Chart.js	4.4.0 — Gráficas del reporte "Acerca del Atlas" (barras, donas, apiladas).
Google Fonts	Sin versión fija — Tipografías Fraunces, DM Sans y Space Mono.

El grafo de la Red de Técnicas es una implementación propia sobre canvas, sin D3 ni Vis.js. Esto reduce las dependencias externas y el peso de carga de la página.

4. Estructura de los archivos CSV

Los tres archivos CSV son el corazón del sistema. La plataforma se actualiza completamente reemplazando estos archivos. A continuación, se documenta cada columna y su uso.

4.1 data_by_technique_id.csv

Una fila por técnica. Constituye la fuente canónica de todos los datos estructurados y cuantitativos. Este archivo se genera mediante el proceso de procesamiento de datos del proyecto, que parte de los registros individuales para producir los agregados por técnica y aplicar la clasificación experta.

Columna	Descripción
Tecnica	Nombre canónico estandarizado de la técnica. Funciona como clave primaria: debe coincidir exactamente con el campo Tecnica de los otros dos CSV.
tecnica_norm	Nombre normalizado alternativo. Se usa como respaldo cuando el campo Tecnica está vacío.
n_fichas	Total de registros individuales que documentan la técnica.
n_mujeres / n_hombres	Desglose de registros por género de quien practica la técnica.
edad_prom	Edad promedio de las y los artesanos que practican la técnica.

estados	Lista de estados separados por coma donde se registró la técnica.
n_man_mano / n_man_telar / n_man_pedal / ...	Número de registros para cada tipo de manufactura.
n_tenido_plantas / n_tenido_minerales / n_tenido_animales / n_tenido_otro	Número de registros para cada tipo de teñido.
n_aprendio_madre / _abuela / _tia / _hermana / _cunada / _instructor / _padre	Conteo de registros donde la persona aprendió de cada figura referida.
n_ens_hijas / _nietos / _sobrinos / _pareja / _estudiantes / _hijos	Conteo de a quiénes enseñan la técnica.
n_no_ha_enseñado	Cuántas y cuántos artesanos no han enseñado aún la técnica.
ceremonia_principal / n_ceremonia_principal / ceremonias_resumen	Ceremonias más comunes asociadas y sus conteos.
prenda_principal / n_prenda_principal / prendas_resumen	Prendas más comunes asociadas y sus conteos.
Materiales_concat_clean	Cadena de materiales utilizados, depurada y concatenada.
CAT-N-1	Categoría experta de nivel 1: Hilados, Teñidos, Tejido, Técnicas decorativas, Acabados.
CAT-N-2	Subcategoría de nivel 2 (por ejemplo: Bordados, Tejidos, Anudados).
CAT-N-3	Tipo de nivel 3.
CAT-N-4	Variante de nivel 4. No todas las técnicas llegan a este nivel.
score_calidad	Indicador numérico que evalúa la riqueza de los reactivos completados (historia, significados, materiales, lengua, multimedia, transmisión, manufactura, teñido, entre otros).
score_volumen	Bonus logarítmico que pondera el número de registros asociados a la técnica.
score_total	Suma de score_calidad y score_volumen. Es el indicador agregado que ordena las técnicas.
ranking	Posición de la técnica en el ranking global de score_total (1 = mejor evaluada).
incluida	Bandera ("sí" / "no") que indica si la técnica forma parte del subset público del Atlas. Determinada en función del ranking, con ajustes editoriales del equipo curatorial.

uuid	Identificador único interno del registro.
count	Campo de conteo auxiliar del procesamiento.

La columna "incluida" es la que determina, a partir de la versión 2.0, qué técnicas aparecen en las vistas públicas de la plataforma. Sustituye al criterio anterior basado en $n_fichas > 1$. El score que la sustenta pondera la calidad informativa de los registros antes que su simple cantidad. El equipo curatorial puede ajustar manualmente el valor de esta columna sobre técnicas específicas para preservar narrativas culturalmente relevantes que el algoritmo no logra capturar por sí solo.

4.2 data_by_record_id.csv

Una fila por ficha individual levantada en campo. Este archivo proviene directamente de KoboToolbox con una transformación mínima. Se utiliza para los campos cualitativos y para calcular las listas únicas (lenguas, municipios) que no están disponibles de manera directa en data_by_technique_id.

Columna clave	Descripción
Tecnica	Nombre de la técnica. Clave de unión con data_by_technique_id.
Sede	Sede donde se levantó la ficha: CDMX, Mérida o Tijuana.
Genero	Género declarado por la persona artesana: mujer, hombre, no especificar.
Edad	Edad de la persona artesana al momento del registro.
Estado / Municipio / Localidad	Ubicación geográfica de la persona artesana.
Lengua	Lengua indígena que habla la persona artesana.
Historia	Texto libre sobre la historia de la técnica. En la ficha se utiliza el fragmento más extenso por técnica.
S1 a S5	Fragmentos de significado cultural. Se utilizan los más extensos como testimonios en la ficha.
Temporalidad	Si la técnica se considera antigua o nueva. Se calcula por mayoría de registros por técnica.
Madre / Abuela / Tia / Hermana / Cunada / Instructora / Padre	Valores binarios (1/0): de quién aprendió la persona artesana. Alimentan las gráficas del reporte.

Hijas / Nietos / Sobrinos / ...	Valores binarios: a quién enseña.
pic / video	Rutas o identificadores de archivos multimedia asociados al registro.
uuid / _index	Identificadores únicos del registro en KoboToolbox.

Las columnas pr1–pr10, c1–c8 y cm1–cm8 son campos internos de KoboToolbox (números de pregunta y respuesta codificada). No son utilizadas directamente por la plataforma.

4.3 indice_imagenes.csv

Una fila por archivo de imagen disponible. Mapea cada imagen a la técnica que representa.

Columna	Descripción
archivo_descargado	Nombre del archivo de imagen tal como se almacena en el servidor. Es la clave que utiliza la plataforma para construir la URL de la imagen.
Tecnica	Nombre de la técnica a la que corresponde la imagen. Debe coincidir con el campo Tecnica de data_by_technique_id.csv.
status	Estado de la descarga del archivo: "ok" o "error". Solo se incluyen las imágenes con status "ok".
Sede / Estado / Municipio	Metadatos geográficos del registro de procedencia.
Genero / tecnica_grupo	Metadatos adicionales del registro.
url / id / error_msg	URL original de KoboToolbox, identificador interno y mensaje de error en caso de aplicar.

4.4 Consistencia entre los CSV

La integridad de la plataforma depende de que los nombres de técnica sean idénticos en los tres archivos. Cualquier discrepancia, incluso de capitalización o de espacios, hará que el cruce falle y que la técnica afectada aparezca incompleta o se duplique.

El proceso recomendado para verificar la consistencia, antes de subir archivos al servidor, consiste en lo siguiente:

- Tomar data_by_technique_id.csv como autoridad: contiene el nombre canónico de cada técnica.
- Comparar contra data_by_record_id.csv y unificar cualquier diferencia mediante un mapeo lower → nombre canónico.

- Verificar lo mismo en indice_imagenes.csv.
- En caso de detectar nombres en los registros que no aparezcan en data_by_technique_id, evaluar si corresponden a técnicas a incorporar o a errores tipográficos a corregir.

En la versión 2.0 se llevó a cabo una unificación masiva de nombres que afectó 107 registros con discrepancias de capitalización. La consistencia entre los CSV es ahora total (158 nombres + el grupo "Otras"). Esta verificación debe repetirse cada vez que se procesen datos nuevos.

5. Cómo actualizar el Atlas

5.1 Actualizar datos de técnicas

Para agregar, modificar o eliminar técnicas del Atlas:

- Generar los nuevos archivos data_by_technique_id.csv y data_by_record_id.csv mediante el proceso de procesamiento de datos del proyecto (Python/Pandas).
- Verificar que los nombres en la columna Tecnica sean consistentes entre ambos archivos y con indice_imagenes.csv (ver sección 4.4).
- Actualizar la columna "incluida" si se desea agregar o retirar técnicas del subset público.
- Subir los nuevos CSV al servidor de UNESCO México, sustituyendo los archivos anteriores en la misma ruta.
- Recargar la plataforma en el navegador (F5). No es necesario modificar ningún archivo HTML ni JavaScript.

Importante: no cambiar los nombres de los archivos CSV. La plataforma los referencia con los nombres exactos: data_by_technique_id.csv, data_by_record_id.csv e indice_imagenes.csv.

5.2 Ajustar el subset de técnicas publicadas

La columna "incluida" de data_by_technique_id.csv es la que determina qué técnicas aparecen en las vistas públicas. Para ajustar el subset sin reprocesar todos los datos, basta con editar manualmente esa columna en el CSV.

El criterio recomendado para tomar decisiones sobre esa columna es:

- Las técnicas con score_total alto (ranking ≤ 60) tienen datos suficientemente sólidos y deben permanecer en el subset.

- Las técnicas en posiciones intermedias del ranking (61–80) pueden incorporarse al subset si poseen valor narrativo o cultural particular, aunque su score sea moderado.
- Las técnicas con score muy bajo o con datos incompletos deben permanecer fuera del subset hasta que se incorporen registros adicionales que las refuercen.

El equipo curatorial conserva, en todo momento, la facultad de aplicar ajustes editoriales sobre esta columna. Los cambios se reflejarán en la plataforma con la siguiente recarga.

5.3 Actualizar la clasificación experta

Las columnas CAT-N-1, CAT-N-2, CAT-N-3 y CAT-N-4 de `data_by_technique_id.csv` son las que determinan la clasificación mostrada en el árbol, en los chips de filtro del mapa y en el catálogo. Para modificar la clasificación de una técnica basta con editar esas columnas en el CSV y subir el archivo actualizado.

Los colores de cada categoría están definidos en el objeto `CAT1_COLOR` al inicio de `main.js`. Si se incorpora una nueva categoría de nivel 1 que no esté en esa tabla, se le asignará el color neutro (arena). Para asignarle un color específico, debe agregarse la entrada correspondiente en ese objeto.

5.4 Agregar o actualizar imágenes

Las imágenes se referencian por nombre de archivo desde `indice_imagenes.csv`. Para agregar imágenes:

- Subir los archivos de imagen al servidor, en la ruta que corresponda.
- Agregar las filas correspondientes en `indice_imagenes.csv` con el nombre exacto del archivo (columna `archivo_descargado`), la técnica asociada (columna `Tecnica`) y status "ok".
- Subir el CSV actualizado al servidor.

La función `imgPath()` de `main.js` construye la URL completa de cada imagen a partir del nombre de archivo. Si las imágenes se mueven a una subcarpeta o a un servidor diferente, debe actualizarse esa función:

```
// Función actual en main.js (línea ~397):
function imgPath(filename) {
  return 'imagenes/' + filename;
}

// Para imágenes alojadas en otro servidor:
```

```
function imgPath(filename) {  
  return 'https://servidor.ejemplo.com/fotos/' + filename;  
}
```

5.5 Modificar textos del reporte

Los textos editoriales (introducciones, callouts, descripción de secciones) se encuentran directamente en el HTML de atlas_textil_v2.html. Para modificarlos basta con editar ese archivo y subirlo al servidor.

Las estadísticas y gráficas del reporte se calculan automáticamente desde los CSV: no requieren edición manual.

6. Datos: arquitectura actual y migración a API

6.1 Arquitectura actual: CSV estáticos

En la actualidad la plataforma no consume ninguna API ni base de datos. Los datos son archivos CSV estáticos almacenados junto con el HTML en el servidor. El navegador los descarga completos al cargar la página.

Esta arquitectura ofrece ventajas claras: no requiere servidor de aplicaciones, ni mantenimiento de base de datos, ni autenticación, ni costos de infraestructura variables. Resulta adecuada para la fase actual del proyecto.

La limitación principal radica en que, para actualizar los datos, debe reemplazarse manualmente los archivos en el servidor.

6.2 Cómo migrar a una API en el futuro

En caso de que en el futuro se decida conectar la plataforma a una API REST (por ejemplo, para actualizaciones automáticas desde KoboToolbox o para vincular una base de datos), el cambio se concentra en un único punto: la función loadCSVs() de main.js.

Actualmente, esa función opera de la siguiente manera:

```
// Arquitectura actual (main.js, ~línea 327):  
async function loadCSVs() {  
  const [techRes, recRes, imgRes] = await Promise.all([  
    fetch('data_by_technique_id.csv'),  
    fetch('data_by_record_id.csv'),  
    fetch('indice_imagenes.csv'),  
  ]
```

```

]);
const techRows = parseCSV(await techRes.text());
const recordRows = parseCSV(await recRes.text());
const imgRows = parseCSV(await imgRes.text());
ATLAS = buildAtlasFromCSV(techRows, recordRows, imgRows);
// ... resto de la inicialización
}

```

Para conectar una API REST que devuelva JSON, se reemplazaría únicamente el bloque de fetch y parseo. La función buildAtlasFromCSV y todo lo demás permanecen iguales, siempre que el JSON devuelto conserve los mismos nombres de campo que los CSV actuales:

```

// Versión con API REST hipotética:
async function loadCSVs() {
  const [tecnicas, registros, imagenes] = await Promise.all([
    fetch('https://api.ejemplo.com/tecnicas').then(r => r.json()),
    fetch('https://api.ejemplo.com/registros').then(r => r.json()),
    fetch('https://api.ejemplo.com/imagenes').then(r => r.json()),
  ]);
  // buildAtlasFromCSV espera arrays de objetos con los mismos nombres
  // de campo que las columnas de los CSV.
  ATLAS = buildAtlasFromCSV(tecnicas, registros, imagenes);
  // ... resto igual
}

```

Si la API devolviera un formato distinto al de los CSV (nombres de campo diferentes, estructura anidada, paginación), tendría que añadirse una función de transformación entre la respuesta de la API y el formato que buildAtlasFromCSV espera. Esa función de transformación sería el único código nuevo necesario.

El diseño actual con buildAtlasFromCSV como capa de abstracción de datos fue pensado precisamente para facilitar esta migración: el resto de la aplicación (mapa, catálogo, red, árbol) no sabe si los datos provienen de CSV o de una API. Solo conoce el objeto ATLAS.

6.3 Consideraciones para una API

Si se decidiera implementar una API en el futuro, conviene tener presentes los siguientes aspectos:

- CORS: la API debe incluir los headers necesarios para permitir consultas desde el dominio de UNESCO México.

- Autenticación: si la API requiere un token, este no puede residir en el código JavaScript del cliente, ya que es público. Se recomienda una API pública de solo lectura o, en su defecto, un proxy del lado del servidor.
- Caching: la plataforma actualmente descarga los CSV completos en cada carga. Una API con soporte de cache HTTP (ETag, Cache-Control) mejoraría el rendimiento para datos que cambian con poca frecuencia.
- Paginación: buildAtlasFromCSV espera todos los datos de una sola vez. Si la API pagina los resultados, deberá agregarse lógica para acumular todas las páginas antes de construir ATLAS.

7. Mantenimiento y diagnóstico

7.1 Verificar que la plataforma carga correctamente

Al abrir la plataforma, si los CSV no se encuentran o no son accesibles, aparece un banner de error rojo en la pantalla de carga con un mensaje descriptivo. Las causas más frecuentes son las siguientes:

- Los archivos CSV no se encuentran en la misma carpeta que index.html.
- Los nombres de los archivos no coinciden exactamente (mayúsculas, guiones bajos, extensiones).
- El servidor no permite acceder a los archivos CSV (permisos o tipo MIME mal configurado).

Para diagnosticar, conviene abrir las herramientas de desarrollador del navegador (F12) y revisar la pestaña "Red" (Network). Si aparecen errores 404 en las peticiones a los CSV, se trata de un problema de ruta. Si aparecen errores 403, el problema está en los permisos del servidor.

7.2 El mapa no muestra los estados de México

El mapa carga el GeoJSON de los estados de México desde un repositorio público de GitHub. En caso de un problema de red o si ese repositorio dejara de estar disponible, el mapa aparecería vacío. Esto no afecta al catálogo ni a las fichas.

La solución de largo plazo consiste en descargar el archivo mexicoHigh.json y alojarlo en el mismo servidor que el Atlas, actualizando la URL en la función initMap() de main.js.

7.3 Actualización de librerías

Las librerías externas (Leaflet, Chart.js, Google Fonts) se cargan desde CDN. Si fuera necesario actualizar a una versión específica, las URLs están en el encabezado de index.html y de atlas_textil_v2.html:


```
<!-- En index.html -->
<link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css"/>
<script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>
```

Se recomienda no actualizar las versiones de las librerías sin verificar previamente que la plataforma sigue funcionando correctamente en un entorno de prueba.

8. Formularios de contribución (KoboToolbox)

La sección "Contribuir" de la plataforma integra tres formularios de KoboToolbox. Las URLs de cada formulario están definidas en el bloque de scripts al final de index.html:

```
// En index.html, al final del body:
const KOBO_URLS = {
  nueva: 'https://ee.kobotoolbox.org/x/GTRuQbob',
  cambio: 'https://ee.kobotoolbox.org/x/q1s1Q5ht',
  comentario: 'https://ee.kobotoolbox.org/x/8V6qx7Dj',
};
```

Si las URLs de los formularios cambian (por ejemplo, en caso de generar una nueva versión del formulario en KoboToolbox), basta con actualizar esos tres valores en index.html y subir el archivo al servidor.

Los datos recopilados por los formularios se almacenan en la cuenta de KoboToolbox del proyecto y son independientes de la plataforma web. El proceso de incorporar nuevas contribuciones al Atlas es manual: requiere revisar la cuenta de KoboToolbox, procesar los datos y actualizar los CSV.

9. Sistema de diseño

La identidad visual de la plataforma está definida en styles.css mediante variables CSS. Los colores principales son:

Variable	Valor y uso
--magenta (#B50552)	Color primario. Encabezados, acentos, botones principales y categoría "Técnicas decorativas".

--azul-mar (#035A79)	Color secundario. Enlaces, elementos interactivos y categoría "Tejido".
--naranja (#FB4801)	Color de acento terciario. Alertas y destacados.
--verde (#05B794)	Categoría "Teñidos".
--ocre (#FFA329)	Categoría "Acabados". Números destacados en el encabezado.
--cian (#00B2C0)	Subcategoría "Tejidos".
--rosa (#DE599B)	Subcategoría "Aplicaciones". Género femenino en gráficas.
--negro (#1A1018)	Fondo del encabezado y de las barras de navegación.
--bg-page (#FFF8F5)	Fondo general de la plataforma.

Las tipografías se cargan desde Google Fonts: Fraunces (títulos y encabezados), DM Sans (cuerpo de texto) y Space Mono (datos numéricos y etiquetas técnicas).